



Sintaxis y Semántica del Lenguaje

Trabajo Práctico Integrador: Diseño e implementación de Lexer y Parser

Intérprete SMART-HOME y Traductor a HTML

Ciclo lectivo: 2026

Equipo docente:

Director de Cátedra

Nombre y Apellido: Ing. Gabriela P. TOMASELLI

Categoría docente: Profesora Asociada

e-mail: gabriela.tomaselli@gmail.com

Docentes

Ing. Gabriela P. TOMASELLI

e-mail: gabriela.tomaselli@gmail.com

Ing. Rodrigo VIGIL

e-mail: rodrigovigil@gmail.com

Ing. Nicolas G. TORTOSA

e-mail: nicotortosa@gmail.com

Ing. Juliana I. TORRE

e-mail: julitorre025@gmail.com



Índice de Contenido

1. Introducción	2
Competencias	3
Objetivo	3
2. Elaboración y entrega	3
Presentaciones.	3
Metodología de entregas parciales y finales.	4
3. Conformación de equipos	4
Identidad	4
Alianza de equipo	4
Matriz de habilidades	4
4. Características de lenguaje SMART HOME	5
Introducción	5
Componentes léxicos o tokens	5
Palabras Reservadas:	5
Identificadores de Sensores, Dispositivos y Atributos	5
Literales y Unidades (Tokens Compuestos)	7
Operadores y Delimitadores	7
E-mail	7
Estructura del documento	8
Listas y objetos del documento	8
Consideraciones:	8
EJEMPLO DE UN ARCHIVO DE TEXTO SMARTHOME	8
5. Traducción a HTML	9
6. Control de errores	9
6.1. Modos de ejecución del intérprete	9
• Modo interactivo - Analiza mientras escribe	10
• Ejecución desde un archivo	10
7. Documentación del trabajo	10
Informe técnico	10
Video Educativo / Promocional:	11
8. Presentación y Criterios de evaluación	11
Documentación: 40 %	11
Funcionamiento del intérprete (software): 60 %	11
9. Ejemplos	12
Básico	12



1. Introducción

Competencias

El presente trabajo práctico integrador pretende desarrollar las siguientes competencias de la asignatura:

- # Capacidad para conocer los fundamentos teóricos de los lenguajes de programación y las técnicas de procesamiento léxico, sintáctico asociadas, y saber aplicarlas para la creación, diseño y procesamiento de lenguajes de programación.

- # Capacidad de diseño, y habilidad instrumental necesaria para llevar a cabo proyectos de cierta envergadura, cuya complejidad exige la utilización de conocimientos adquiridos en diversas asignaturas a lo largo de la carrera.

- # Aptitud y actitud para trabajar en equipo.

Objetivo

Una empresa de domótica quiere que los usuarios configuren sus casas usando scripts sencillos en lugar de aplicaciones complejas. Para ello deberán diseñar y construir los analizadores léxico, sintáctico y semántico que permita analizar, validar scripts que contienen reglas de automatización ejecutables por un controlador central (no implementado) además de generar una interfaz web que representa el estado y control de una casa inteligente en tiempo real.

Este trabajo se enfoca en el Internet de las Cosas (IoT) y Domótica. introduce el concepto de eventos y acciones, lo que permite trabajar con una estructura de control muy clara.

A diferencia de los lenguajes de programación tradicionales (ejecución secuencial), Smart Home es un lenguaje basado en eventos, con un ejecución reactiva (el programa espera a que algo pase).

El compilador a construir recibe un archivo de texto (script) conteniendo detección de eventos condicionales o iterativos y acciones a realizar cuando se cumplen las mismas y deberá indicar si está bien construido (adecuado al formato indicado, sin errores) de otra manera indicar los errores; adicionalmente a medida que analiza el script deberá transformar el contenido en un documento HTML.

Hay dos grupos de herramientas que se pueden usar para generar los analizadores:

El analizador lexicográfico es un módulo que recibe una secuencia de caracteres que componen el documento a analizar y lo convierte lógicamente en una secuencia de tokens. El analizador sintáctico recibe la secuencia de tokens que le entrega el analizador lexicográfico y verifica si es correcta de acuerdo a una determinada gramática. Adicionalmente puede generar diversas salidas de acuerdo a criterios especificados.

1) Se utilizan expresiones regulares y autómatas finitos para el análisis lexicográfico y la técnica LALR para el análisis sintáctico. Ejemplos de esto son lex y yacc, que generan código C o C++, o JLex y CUP, que generan código Java. flex y bison son implementaciones libres y gratuitas de lex y yacc.

2) El otro utiliza la técnica LL(k) tanto para el análisis léxico como para el sintáctico, generando parsers descendentes recursivos. Ejemplos son JavaCC, que genera código Java, y ANTLR, que está escrito en Java pero puede generar código Java, C++ python o C#.

https://en.wikipedia.org/wiki/Comparison_of_parser_generators

2. Elaboración y entrega

Presentaciones.

Se prevén cuatro instancias de presentación:

Entrega inicial: Conformación de equipo (matriz, identidad, alianza).

Domingo 12 de Abril de 2026

-1er entrega: Primera documentación técnica del proyecto; gramática a generar.



Domingo 03 de Mayo de 2026

. **2da entrega:** Presentación del lexer o scanner que reconozca los tokens del lenguaje.

Domingo 07 de Junio de 2026

. **3ra entrega final:** Presentación de trabajo completo. Lexer y parser, incluye la presentación del trabajo ante la clase con una exposición de no más de 20 minutos.-

Domingo 05 de Julio de 2026

Metodología de entregas parciales y finales.

o Un archivo comprimido deberá ser "subido" a la tarea correspondiente dentro del curso en el campus virtual.

o Dicho archivo comprimido deberá contener, según corresponda:

directorio **doc**. Documentación del trabajo (véase el apartado n° 6)

directorio **src**. archivos fuente del proyecto (flex, bison, python, archivos de C (".c", ".h")

directorio **bin**. Archivos binarios listos para ser ejecutados. (lexer y parser)

directorio **prueba**. Archivos de ejemplo de pseudocódigo con la extensión ".json"

3. Conformación de equipos

Todos los alumnos deberán completar la [encuesta inicial 2026](#), a fin de conocer habilidades y capacidades de cada alumno, la misma estará disponible hasta el día *28 de Marzo de 2026*, luego se publicarán los resultados, a fin de que cada equipo pueda conformarse de la mejor manera.

El trabajo se realizará de forma grupal en equipos de entre cinco y seis alumnos (**5-6**), donde cada integrante deberá realizar y presentar una parte del trabajo durante la presentación final.

Cada equipo debe ser responsable, autónomo y adaptable.

Cada equipo tendrá la libertad de definir cómo trabajan, pero debe haber un líder que de las pautas de qué se debe hacer, y en qué condiciones se espera que se haga o cuales son los resultados esperados, estará representado por un delegado, encargado de enviar las entregas parciales y el trabajo final al campus virtual, enviará consultas o dudas, etc.

Cada equipo tendrá un docente designado como tutor encargado del seguimiento del trabajo, responderá inquietudes.

Identidad

Cada equipo deberá proponer un nombre y logo para identificarse . Algo que represente al equipo y lo haga diferente a los demás.

Un equipo sin identidad nunca será un gran equipo y puede, incluso, que no sea ni equipo.

Alianza de equipo

Un breve enunciado donde el equipo define como trabajará y se organizará, a que se comprometen, cuales son sus metas y valores , los objetivos comunes.

Por ejemplo, si el trabajo requerirá de juntarse una vez a la semana fuera de la universidad, establecer en qué días y horarios podría ser, o cómo será la organización de esa reunión. Y si necesitaremos comunicarnos fuera de clase, cuáles serán esos canales.

Matriz de habilidades

Cada equipo deberá conformar su propia **matriz de habilidades**:



Deberán armar una tabla y escribir en las columnas las principales competencias necesarias para completar el trabajo. En las filas de esa misma tabla los nombres de las personas del equipo.

Luego:

Puntúa dentro de la tabla el grado de habilidad de cada persona respecto a cada competencia:

→ Poner un asterisco * o estrella ★ en donde la persona es experta.

→ Poner un punto ○ donde la persona no es experta pero podría desenvolverse con dicha competencia (le interesa, es bueno en eso aunque no es su mayor interés, o está dispuesto a aprenderlo).

Dejar en blanco competencias que una persona desconozca por completo (o se niegue a participar en ellas).

Ejemplo de Competencia: Inglés, buscar info, programar, idear soluciones, escribir documentación, redes sociales, edición video, cebar mates, etc.-

4. Características de lenguaje SMART HOME

Introducción

Smart Home es un lenguaje de programación orientado a eventos que controla una casa inteligente.

Su función es reconocer, validar y transformar instrucciones de texto sencillas ("Si la temperatura sube, enciende el aire") en una interfaz web funcional o en un sistema lógico de control.

El usuario escribe reglas de automatización en un archivo de texto. El intérprete procesa ese texto y genera un Dashboard HTML/JS donde se puede ver el estado de los dispositivos.

Componentes léxicos o tokens

Palabras Reservadas:

Son términos fijos que definen la estructura de control del lenguaje. No pueden usarse como nombres de dispositivos.

- WHEN : Inicia un bloque disparado por evento.
- IF / THEN / ELSE : Estructuras condicionales.
- DO : Delimitador de inicio de acción.
- END : Finalización o cierre de bloque de acción
- EVERY : Define una tarea programada por tiempo.

Observaciones

No se distinguirá entre mayúsculas ni minúsculas.

Las palabras reservadas no se podrán utilizar como identificadores.

Identificadores de Sensores, Dispositivos y Atributos

Representan los objetos físicos y sus propiedades.

- **Sensores** (Dispositivos de Entrada) : Son dispositivos de "solo lectura" que proporcionan los datos para las condiciones de los bloques `WHEN` e `IF`.

sensor_temp	Temperatura	°C	-10.0 a 50.0	Temperatura dentro de la casa.
sensor_humedad	Humedad	%	0% a 100%	Nivel de humedad ambiental.
sensor_luz	Iluminancia	lux	0 a 1000	Nivel de luz.
sensor_movimiento	Presencia	BOOL	TRUE/FALSE	Detecta si hay alguien en la sala.
sensor_humo	Seguridad	BOOL	TRUE/FALSE	Detector de incendios.



- **Actuadores** (Dispositivos de Salida) : Nombres que el usuario da a sus aparatos a controlar y realizan una acción física tras recibir un comando. Tienen atributos que pueden ser modificados.
 - Deben empezar con prefijo común que identifica el actuador, seguido por un guión bajo y luego pueden contener letras, números o más guiones bajos.
 - El diseño debe permitir tanto la **lectura** (en condicionales) como la **escritura** (en acciones) mediante una estructura clara de nombres de atributos y unidades, para ello debe implementar la **notación de punto**. Esto permite separar el "Sujeto" (dispositivo) de su "Característica" (atributo).
 - **Sintaxis:** `dispositivo.atributo = valor`
 - Atributos: Propiedades accesibles mediante la notación de punto (ej. ``brillo``, ``estado``). Esta estructura define qué es legal y qué no dentro del lenguaje **Smart Home**.

Dispositivo	Atributo	Tipo de Dato	Rango / Valores	Permisos
foco_	.estado	BOOL	ON, OFF	Lectura/Escritura
	.brillo	PERCENT	0% a 100%	Lectura/Escritura
	.color	NOMBRE	blanco, rojo, azul	Lectura/Escritura
aire_	.estado	BOOL	ON, OFF	Lectura/Escritura
	.modo	DISCRETO	FRIO,CALOR,VENT	Lectura/Escritura
	.temp_obj	TEMP	16 °C a 30 °C	Lectura/Escritura
	.temp_act	TEMP	-10 °C a 50 °C	Solo Lectura
persiana_	.posición	PERCENT	0% a 100%	Lectura/Escritura
cerradura_	.estado	BOOL	ON, OFF	Lectura/Escritura
reloj_	.hora	TIME	00:00 a 23:59	Solo Lectura
	.fecha	DATE	01/01/2000 a 31/12/2099	Solo Lectura
altavoz_	.volumen	PERCENT	0% a 100%	Lectura/Escritura
	.mute	BOOL	ON, OFF	Lectura/Escritura
	.mensaje	texto		Lectura/Escritura
	.email_notif	email		Lectura/Escritura
alarma_	.estado	BOOL	ON, OFF	Lectura/Escritura
	.activada	BOOL	ON, OFF	Lectura/Escritura



Literales y Unidades (Tokens Compuestos)

Este es un punto interesante del proyecto, ya que algunos de los valores que el lexer debe capturar tienen contexto:

- texto: cualquier cadena de alfanuméricos y caracteres especiales. A utilizarse entre comillas.
- Booleanos: `TRUE`, `FALSE` en sensores, `ON`, `OFF` en dispositivos
- Numéricos con Unidad: El lexer debe identificar el número y además la unidad de medida.
 - TEMPERATURA: expresada en grados centígrados Ej. `25°C`, `-5°C`.
 - PORCENTAJE: acompañados del símbolo % Ej. `50%`, `100%`.
 - TIEMPO: segundos, minutos, u horas Ej. `10s`, `5m`, `1h`.
 - ILUMINANCIA: Ej. `600lux` 0 a 1000
- HORA: Ej. `06:00`, `23:15` (debe respetar el formato `HH:MM` de 24 horas).
- FECHA: Ej: 21/04/2026 (debe respetar el formato DD/MM/AAAA)
 - A los efectos de este práctico limitaremos las fechas de la siguiente manera
 - Año debe ser un número entre 1900 y 2099.
 - Mes debe ser un número entre 1 y 12
 - Día debe ser un número entre 1 y 31
 - No es necesario validar correspondencia entre día y mes (ej. 31 de feb).
- Email

Operadores y Delimitadores

- Comparación: `==` (igualdad), `!=` (diferente), `>`, `<`, `>=`, `<=`.
- Lógicos: `AND`, `OR`, `NOT`.
- Asignación: El signo `=` para cambiar estados (ej. `brillo = 50%`).
- Puntuación: El punto `.` para acceder a atributos

E-mail

Un correo electrónico es una secuencia de caracteres que describe el patrón de un correo electrónico válido.

El formato de un correo electrónico válido es: **Nombre de usuario, Símbolo arroba (@), Dominio, Extensión.**

En esta expresión, los caracteres que se pueden usar para el *nombre y dominio* son: Letras , Dígitos , Guiones bajos , Puntos , Signos de más (+), Signos de menos (-)

El símbolo @ es obligatorio en todas las direcciones de correo electrónico válidas.

la *Extensión* admite letras de longitud 2 hasta 4.



Estructura del documento

Listas y objetos del documento

- Una línea de comentario puede aparecer en cualquier lado.
- El programa está compuesto de una o más instrucciones (When / Every / Condicional / Asignación)
- Los bloques de control deben respetar su formato
 - WHEN <condición> DO <acción> END
 - EVERY <tiempo> DO <acción> END
 - <acción> puede ser una asignación o un condicional.
- Condicional
 - IF <condición> THEN <acción> ELSE <acción>
 - <condición> debe dar un resultado TRUE o FALSE, puede ser un booleano, una comparación, una negación (NOT) o varias condiciones agregadas con AND u OR.
 - <acción> puede ser una asignación o un condicional.
- Asignación
 - dispositivo.atributo = valor

Consideraciones:

- Por simplicidad no se contempla dentro del documento vocales acentuadas ni eñe (ñ).
- // indica que es un comentario, debe omitirse en gramática.

EJEMPLO DE UN ARCHIVO DE TEXTO SMARTHOME

```
// El sensor de luz (entrada) activa el foco (salida)
WHEN sensor_luz < 250lux DO
    foco_entrada.estado = ON
    foco_entrada.brillo = 80%
    foco_patio.estado = ON
    foco_patio.color = blue
END

// Gestión inteligente del aire acondicionado
WHEN sensor_movimiento == TRUE DO
    IF sensor_temp_int > 26°C THEN
        aire_acondicionado.estado = ON
        aire_acondicionado.modos = FRIO
        aire_acondicionado.temp_objetivo = 22°C
    ELSE
        aire_acondicionado.estado = OFF
    END
END

EVERY 30m DO
    IF reloj.hora > 22:00 AND alarma.estado == OFF THEN
        persiana_sala.posicion = 0%
```




```
    cerradura_principal.estado = ON
    altavoz_comedor.volumen = 0%
    altavoz_comedor.mensaje = "Son las 22hs, hora de dormir"
    altavoz_comedor.email = felipe@smart-home.com.ar
END
END
IF sensor_humo == TRUE AND aire_acondicionado.estado == OFF THEN
    aire_acondicionado.estado = ON
    aire_acondicionado.modo = FRIO
    aire_acondicionado.temp_objetivo = 22°C
    cerradura_principal.estado = OFF
    persiana_comedor.posicion = 100%
    altavoz_comedor.mensaje = "PELIGRO. HUMO DETECTADO"
    altavoz_comedor.email = bomberos@smart-home.com.ar
END
```

5. Traducción a HTML

Traducir el documento, generando un archivo de texto HTML ,transformando algunos elementos en etiquetas HTML. Formateado de la siguiente manera:

- Nombre de archivo: igual al archivo fuente pero con extensión HTM o HTML.
- Crear un **<div>** con un borde de 1px verde y un padding de 20px; para indicar estado de sensores.
- Sensores: Nombre del sensor encerrados entre tags: **<h2>**, junto con su valor y unidad.
- Actuadores: cada actuador debe contenerse en un div **<div>** con un borde de 1px gris y un padding de 20px;.
- Nombre del actuador : encerrados entre tags: **<h1>**
- Atributos de actuadores : deben traducirse como listas html **ul** y los valores de atributos como item **li**
- Todos items **EMAIL** deberá traducirse como tag html link **a**, con href mailto y el texto "contactar a <nombre de usuario>". Ejemplo `<a href=mailto:"disha@uplers.com">Contactar a disha</a>`

6. Control de errores

El intérprete deberá detectar, capturar y controlar errores indicando tipo de error , número de línea y cadena que generó el error.

- Léxicos:
 - Etiquetas faltantes o mal escritas.
 - Utilización de símbolos no permitidos.
 - Etc.
- De ejecución
 - archivo de entrada inexistente o con una extensión incorrecta.
 - Etc.

6.1. Modos de ejecución del intérprete

El intérprete deberá ejecutarse de dos formas diferentes:



• **Modo interactivo - Analiza mientras escribe**

- Se escribe e interpreta manualmente desde un terminal de texto.
- Útil para reconocer tokens, depuración y pruebas.
- **Obligatorio solo para tercera presentación (lexer)**

• **Ejecución desde un archivo**

- Se interpretarán las sentencias de un archivo pasado como argumento desde la línea de comandos, o abierto por opciones gráficas.
- El archivo deberá tener la extensión ".smart"
- Opcional para entrega de Lexer
- **Obligatorio para presentación final (parser)**

7. Documentación del trabajo

Se deberá documentar todo el desarrollo de trabajo; recolectando, redactando, describiendo y exponiendo todo lo realizado.

Para ello deberán realizar y presentar:

Informe técnico

Documento principal del trabajo, que resume lo realizado desde el inicio, desde la conformación del grupo hasta la entrega final y exposición. Escrito con sus palabras (No solo wikipedia, ChatGPT), resumiendo y promocionando el trabajo desarrollado, con las siguientes características:

- Portada
 - o Título del proyecto desarrollado
 - o Nombre y apellidos de las personas que forman el grupo
 - o Nombre de la asignatura
 - o Nombre de la carrera
 - o Primer cuatrimestre
 - o Curso académico: 2026
 - o Universidad y regional
 - o Lugar y fecha
- Índice
 - o Las páginas deberán estar numeradas.
- Introducción
 - o Breve descripción del trabajo realizado y de las partes del documento.
 - o Descripción de cómo se implementó la solución.
 - o Información y requerimientos de software para ejecutar y compilar el tp (versiones de compiladores, herramientas, plataforma, etc).
- Conformación del Grupo: Integrantes, matriz, identidad, alianza.
- Gramática.
 - o Descripción de cada regla de la gramática libre de contexto a utilizar.
 - # Símbolos de la gramática
 - # Símbolos terminales (componentes léxicos)
 - # Símbolos no terminales
 - # Reglas de producción de la gramática
 - # Se valorará la inclusión de gráficos explicativos.
- Análisis léxico
 - o Descripción del archivo del lexer para definir y reconocer los componentes léxicos.
- Análisis sintáctico:
 - o Descripción del archivo del parser utilizado para definir la gramática libre de contexto.
- Funciones auxiliares



- o Se deben indicar y describir las funciones auxiliares que se hayan codificado.
- Modo de obtención del intérprete
 - o Nombre y descripción de cada archivo utilizado
- Modo de ejecución del intérprete
 - o Interactiva
 - o A partir de un archivo
- Conclusiones:
 - o Reflexión sobre el trabajo realizado.
 - o Puntos fuertes y puntos débiles del intérprete desarrollado.
- Bibliografía o referencias web.
- Anexos
 - o Se podrían incluir aquellos anexos que se consideren oportunos para mejorar la calidad de la documentación

Video Educativo / Promocional:

Resumen audiovisual de hasta 7 minutos, relatando y presentando con sus palabras el trabajo realizado por el equipo.

Debe resumir y destacar lo documentado en el Informe final.

Debe ser publicado de manera permanente en alguna plataforma de videos (youtube, vimeo, tiktok, etc)

8. Presentación y Criterios de evaluación

Documentación: 40 %

Se tendrá en cuenta lo indicado en el apartado no 7.

- o El código elaborado deberá estar documentado.
- o Se valorará la inclusión de gráficos o figuras.
- o Se valorará la cantidad, originalidad y complejidad de los ejemplos propuestos.
- o Debe irse actualizando con cada entrega
- o Conocimiento que cada integrante del grupo aporte durante la exposición final del trabajo.

Funcionamiento del intérprete (software): 60 %

o El intérprete deberá:

funcionar correctamente tanto de forma interactiva como ejecutando las instrucciones desde los archivos de ejemplo según la entrega

Indicar como salida si el análisis fue exitoso (archivo correctamente codificado, sin errores) , en otro caso indicar los errores existentes (indicando tipo de error , número de línea y cadena que generó el error)

Traducir el documento, generando un archivo de texto HTML correcto.

en particular, deberá evaluar y traducir correctamente el ejemplo propuesto por el grupo y dos ejemplos más: uno al azar de los presentados por otros grupos y otro ejemplo elaborado por la cátedra.

o Se valorará

la completitud del lenguaje de código.

La calidad en el diseño del lenguaje y la gramática.

El control de errores.

La ampliación de elementos del lenguaje.

Observación:

Además, se valorará la asistencia a clase de prácticas y la resolución de dificultades encontradas durante la elaboración del trabajo.



9. Ejemplos

Básico

```
// El sensor de luz (entrada) activa el foco (salida)
WHEN sensor_luz < 250lux DO
    foco_entrada.estado = ON
    foco_entrada.brillo = 80%
    foco_patio.estado = ON
    foco_patio.color = blue
END

// Gestión inteligente del aire acondicionado
WHEN sensor_movimiento == TRUE DO
    IF sensor_temp_int > 26°C THEN
        aire_acondicionado.estado = ON
        aire_acondicionado.modo = FRIO
        aire_acondicionado.temp_objetivo = 22°C
    ELSE
        aire_acondicionado.estado = OFF
    END
END

END

EVERY 30m DO
    IF reloj.hora > 22:00 AND alarma.estado == OFF THEN
        persiana_sala.posicion = 0%
        cerradura_principal.estado = ON
        altavoz_comedor.volumen = 0%
        altavoz_comedor.mensaje = "Son las 22hs, hora de dormir"
        altavoz_comedor.email = felipe@smart-home.com.ar
    END
END

END

IF sensor_humo == TRUE AND aire_acondicionado.estado == OFF THEN
    aire_acondicionado.estado = ON
    aire_acondicionado.modo = FRIO
    aire_acondicionado.temp_objetivo = 22°C
    cerradura_principal.estado = OFF
    persiana_comedor.posicion = 100%
    altavoz_comedor.mensaje = "PELIGRO. HUMO DETECTADO"
    altavoz_comedor.email = bomberos@smart-home.com.ar
```



END